

AgenticPD 项目报告

目录

AgenticPD 项目报告

1. 环境与依赖
 - 1.1 必需运行环境
 - 1.2 Python 依赖
 - 1.3 运行前检查
2. 快速开始
 - 2.1 安装位置与目录
 - 2.2 原生多 Agent 优化
 - 2.3 Doomed/GWTW Demo
 - 2.4 查看结果与生成可视化
3. 项目核心机制
 - 3.1 原生多 Agent 调参线
 - 3.2 Doomed/GWTW 调度线
 - 3.3 Workflow
 - 3.4 Architecture
4. Data Model 与对外接口
 - 4.1 实验前参数在哪里配置
 - 4.2 主要 Data Model
 - 4.3 对外接口
 - 4.4 实验完成后结果在哪里
5. 项目不足与后续改进方向
 - 5.1 当前工程不足
 - 5.2 当前算法不足
 - 5.3 系统与实验不足
 - 5.4 建议的后续优先级
6. 我的理解

AgenticPD 项目报告

本文基于当前仓库实现整理。AgenticPD 是位于 OpenROAD-flow-scripts (ORFS) 之上的物理设计 QoR 优化实验框架，核心目标是把参数生成、阶段执行、分支复用、决策记录和最终 QoR 组织成可复现、可观察的 Trial。

1. 环境与依赖

1.1 必需运行环境

1.1.1 操作系统与基础工具

建议使用 ORFS 官方支持的 Linux 环境；Windows 用户应通过 WSL2 使用。基础环境 至少需要：

- 64 位 Linux 或 WSL2；
- Bash 与 GNU Make；
- Python 3.10 或更新版本；
- `pip3`；
- 可正常运行的 OpenROAD-flow-scripts；
- 目标 platform 对应的 OpenROAD、PDK 和 design files。

AgenticPD 不单独安装 OpenROAD 或 PDK，而是直接复用外层 ORFS 环境。当前 Demo 配置使用 `sky130hd/gcd`，因此应先确认 `flow/designs/sky130hd/gcd/config.mk` 存在，并能由 ORFS baseline 正常执行。

1.1.2 目录约束

AgenticPD 依赖与 ORFS 的相对目录关系，项目目录应为：

```
<ORFS_ROOT>/flow/agenticpd/
```

`config.py` 通过自身位置推导 `flow/`、`runs/`、ORFS reports 和 results 路径。如果把项目放在 ORFS 根目录之外，真实 `ORFSRunner` 将无法按当前路径契约工作。

1.2 Python 依赖

1.2.1 requirements.txt

根目录 `requirements.txt` 是唯一 Python 依赖清单：

包	最低版本	用途
<code>openai</code>	1.0	调用 OpenAI-compatible LLM API
<code>matplotlib</code>	3.5	生成原生优化树图片
<code>PyYAML</code>	6.0	解析 Doomed/GWTW 实验 YAML

安装命令：

```
cd <ORFS_ROOT>/flow/agenticpd
pip3 install -r requirements.txt
```

Python 标准库负责 JSON、dataclass、path、subprocess、hash 和 unittest 等功能，不需要在 `requirements.txt` 中重复声明。

1.2.2 LLM 与网络条件

真实 Agent 模式需要：

- 能访问配置的 OpenAI-compatible API endpoint;
- `.env` 或环境变量中的 `DEEPSEEK_API_KEY` ;
- `config.py::FrameworkConfig` 中与服务匹配的 base URL 和 model name。

`--mock-llm` 不访问网络，`--mock-orfs` 不调用 EDA。两者同时使用时只验证软件 控制流，不能证明真实 LLM 或真实物理设计结果。

1.3 运行前检查

1.3.1 软件检查

```
python3 --version
python3 -c "import openai, matplotlib, yaml; print('Python dependencies: OK')"
test -f ../Makefile
test -f ../designs/sky130hd/gcd/config.mk
make check
```

正式实验还应检查 `environment_manifest.json`，确保 ORFS、OpenROAD 和 PDK revision 不再是 `unresolved`。Python 检查通过不等于 ORFS 可运行，真实实验前 仍需执行第 2.1.3 节的 baseline。

2. 快速开始

2.1 安装位置与目录

2.1.1 放置目录

项目必须放在 ORFS 的 `flow/` 目录下，且不要再多套一层目录：

```
<ORFS_ROOT>/
└─ flow/
    ├── designs/
    ├── Makefile
    └─ agenticpd/
        ├── main.py
        ├── multi_agent_gwtw.py
        ├── configs/
        └─ requirements.txt
```

下文除 ORFS 基线检查外，默认从 `<ORFS_ROOT>/flow/agenticpd` 执行命令。代码中的路径由 `config.py` 相对自身位置推导，不应写死用户目录。

2.1.2 安装 Python 依赖

```
cd <ORFS_ROOT>/flow/agentcpcd
pip3 install -r requirements.txt
cp .env.example .env
```

真实 LLM 模式需要在 `.env` 中配置 `DEEPSEEK_API_KEY`。`.env`、token 和 密钥不得进入 Git 或日志。纯 Mock 模式不需要 API key。

2.1.3 检查 ORFS 基线

在运行真实优化前，先证明 ORFS 和目标 PDK 本身可用：

```
cd <ORFS_ROOT>/flow
make DESIGN_CONFIG=./designs/sky130hd/gcd/config.mk
```

如果基线失败，应先修复 ORFS、OpenROAD 或 PDK 环境，而不是调整 Agent prompt。

2.2 原生多 Agent 优化

2.2.1 零 LLM、零 EDA 调试

```
python3 main.py --mock-llm --mock-orfs --iterations 5
```

该命令验证 `JudgeAgent`、四个 `StageAgent`、优化树、Trial 持久化和主循环，但 QoR 是合成值，不能作为真实实验结果。

2.2.2 真实 ORFS 基线

```
python3 main.py \
  --baseline-only \
  --platform sky130hd \
  --design gcd
```

该命令不调用 LLM，只执行一次真实 ORFS baseline 并解析最终 QoR，适合检查 “Python → make → ORFS report → QoR parser” 链路。

2.2.3 真实多 Agent 优化

```
python3 main.py \
  --platform sky130hd \
  --design gcd \
  --iterations 3
```

每轮由 `JudgeAgent` 选择优化树节点和分支阶段，FP、PL、CTS、RT `StageAgent` 生成对应阶段参数，`Optimizer.run_iteration()` 调用 `ORFSRunner` 执行并更新当前最佳 QoR。

2.2.4 恢复与只解析

```
# 恢复同一 platform/design 最近一次 session
python3 main.py --platform sky130hd --design gcd --resume latest

# 不运行 EDA, 只解析一个已有 ORFS variant
python3 main.py --parse-only <variant>
```

`--resume` 只用于原生入口。`--parse-only` 适合验证已有 `6_report.json`、`6_finish.rpt` 或 `6_report.log` 的解析结果。

2.3 Doomed/GWTW Demo

2.3.1 离线闭环

```
python3 multi_agent_gwtw.py \
  --config configs/experiments/multi-agent-gwtw-demo.yml \
  --mock-llm \
  --mock-orfs
```

该命令沿用与真实 Demo 相同的 orchestration, 只替换 LLM client 和 ORFS runner。它用于快速检查 population、PL/CTS 决策、pause、audit、fork、checkpoint resolution、finish 和 trace。

2.3.2 真实闭环

```
python3 multi_agent_gwtw.py \
  --config configs/experiments/multi-agent-gwtw-demo.yml
```

该命令使用真实 Judge + 四个 Stage Agent 和真实 ORFS。运行前应冻结 YAML, 运行后不能只看终端 `complete` 摘要, 还要检查 finish Trial 的 `final_qor`、`report_path` 和 ORFS 原始 post-route report。

2.4 查看结果与生成可视化

2.4.1 生成静态 HTML

```
python3 tools/session_visualize.py \
  runs/sky130hd_gcd/<session>
```

输出位于:

```
runs/sky130hd_gcd/<session>/visualization/index.html
runs/sky130hd_gcd/<session>/visualization/session_data.json
```

`index.html` 可直接由浏览器打开, 不需要 HTTP server、CDN 或网络。

2.4.2 查看 Trial 与安全清理

```
python3 tools/trial_inspect.py --list sky130hd gcd
python3 tools/trial_inspect.py <trial_id> --stages
python3 tools/clean.py sky130hd gcd --dry-run
```

`clean.py` 会涉及真实运行产物，必须先执行 `--dry-run` 并核对目标，不能对 宽泛目录做递归删除。

2.4.3 运行纯 Python 检查

```
make check
```

如果本机还保留不进 Git 的 `tests/`，再运行：

```
make test
```

3. 项目核心机制

3.1 原生多 Agent 调参线

3.1.1 Agent 分工

原生机制由一个 `JudgeAgent` 和四个 `StageAgent` 组成：

- `JudgeAgent.act()` 读取历史和 `OptimizationTree`，输出 branch node、branch stage 及给各 Stage Agent 的 hint。
- `FPAgent`、`PLAgent`、`CTSAgent`、`RTAgent` 只负责自己阶段的参数。
- `StageAgent.validate()` 按 `config.py::PARAM_SPACE` 做类型、范围和名称校验。
- `LLMClient.chat_json()` 提供真实模型调用；`MockLLMClient` 提供确定性调试。

Agent 只提出参数，不直接运行 ORFS，也不直接决定一个 checkpoint 是否安全。

3.1.2 优化树与分支执行

`Optimizer.run()` 先执行 baseline，再循环调用 `run_iteration()`。优化树节点记录 stage、parent、参数快照、QoR、branch count 和 `source_trial_id`。Judge 选择某个节点后，系统构造继承参数，并由 `resolve_checkpoint()` 判断：

- checkpoint manifest 和参数均兼容： `checkpoint_fork`；
- 最深 checkpoint 不兼容但祖先 checkpoint 可用：向更早阶段回退；
- 没有合法 checkpoint： `full_restart`，从 FP 重新执行。

因此，“Judge 请求从 CTS 开始”不等于“实际一定从 CTS 开始”，实际起点以 `ExecutionResolution.effective_start_stage` 为准。

3.1.3 QoR 更新

`ORFSRunner.run_stage()` 返回阶段 `StageResult` , `run_finish()` 从 post-route 报告读取最终 QoR。
`utils.qor_is_better()` 采用 timing-first 比较规则更新最佳 候选。中间阶段 WNS/TNS 只能辅助决策, 最终胜负仍以 finish QoR 为准。

3.2 Doomed/GWTW 调度线

3.2.1 Population 与决策屏障

`MultiAgentGWTWOrchestrator` 根据 YAML 创建初始 population。每个候选都调用 Judge 和 FP、PL、CTS、RT Agent 形成完整参数提案, 随后先执行到 PL。PL cohort 全部到达屏障后统一决策; 进入 CTS 后再次形成 cohort 并统一决策。

这种机制与原生逐 iteration 优化不同: 它关注同一阶段的一组候选, 而不是只对 一个候选立即进入下一阶段。

3.2.2 Doomed 分类

`build_minimal_observation()` 从 Trial 和 stage checkpoint 构造 `MinimalObservation` , 其中包含阶段 WNS/TNS、状态、耗时、失败类型和 lineage。 `doomed_predictor.predict()` 输出:

- `hard_dead`: 执行失败、关键证据缺失等, 不能继续;
- `soft_bad`: 当前 cohort 内相对较差, 通常 pause;
- `survivor`: 当前 cohort 内相对较好, 可继续或作为 fork parent。

`risk_score` 是同一 cohort 内的相对排序, 不是经过校准的失败概率。

3.2.3 GWTW 资源分配

`gtw_scheduler.schedule()` 根据 survivor 数量、audit quota、population size 和 seed 输出 `continue`、`pause`、`audit_continue` 以及 `ForkRequest`。GWTW 不生成参数; child 参数由 Stage Agent 和 mutation planner 提供。

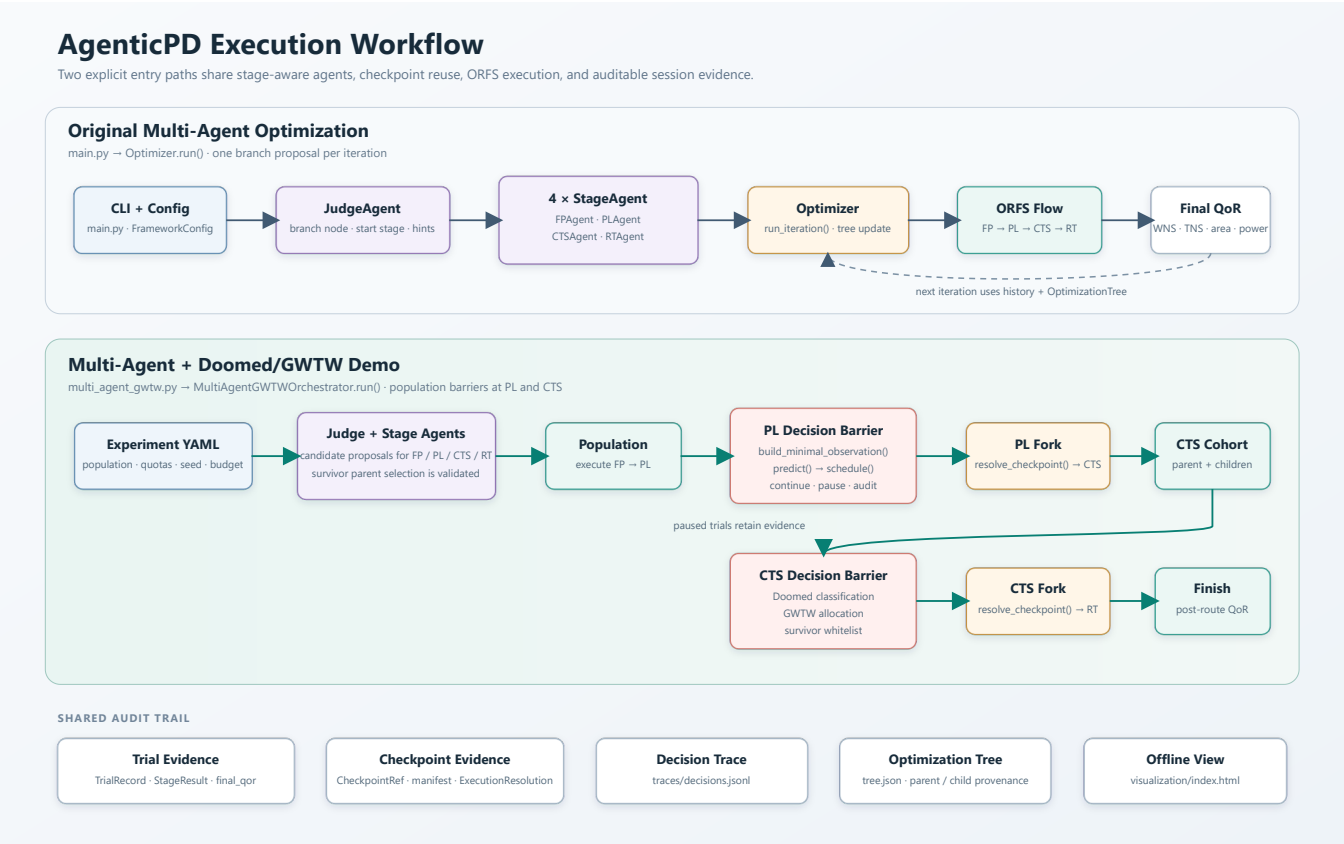
`plan_cohort()` 将 observation、DoomedDecision、GWTWDecision 和 fork plan 组装成纯数据计划, `execute_cohort()` 负责落盘、幂等恢复和 checkpoint resolution。Judge 选择 parent 时只能落在 survivor whitelist, 越界选择必须 拒绝或使用带 trace 的确定性 fallback。

3.2.4 Pause、Audit 与 Finish

pause Trial 保留 checkpoint、决策和 trace, 但没有 `final_qor`。 `audit_continue` 允许少量相对较差候选继续运行, 用于观察 Doomed 是否误判。CTS survivor 和 audit candidate 可进入 RT/finish; 只有真实 finish 且 QoR 完整的 Trial 才能进入最终结果。

3.3 Workflow

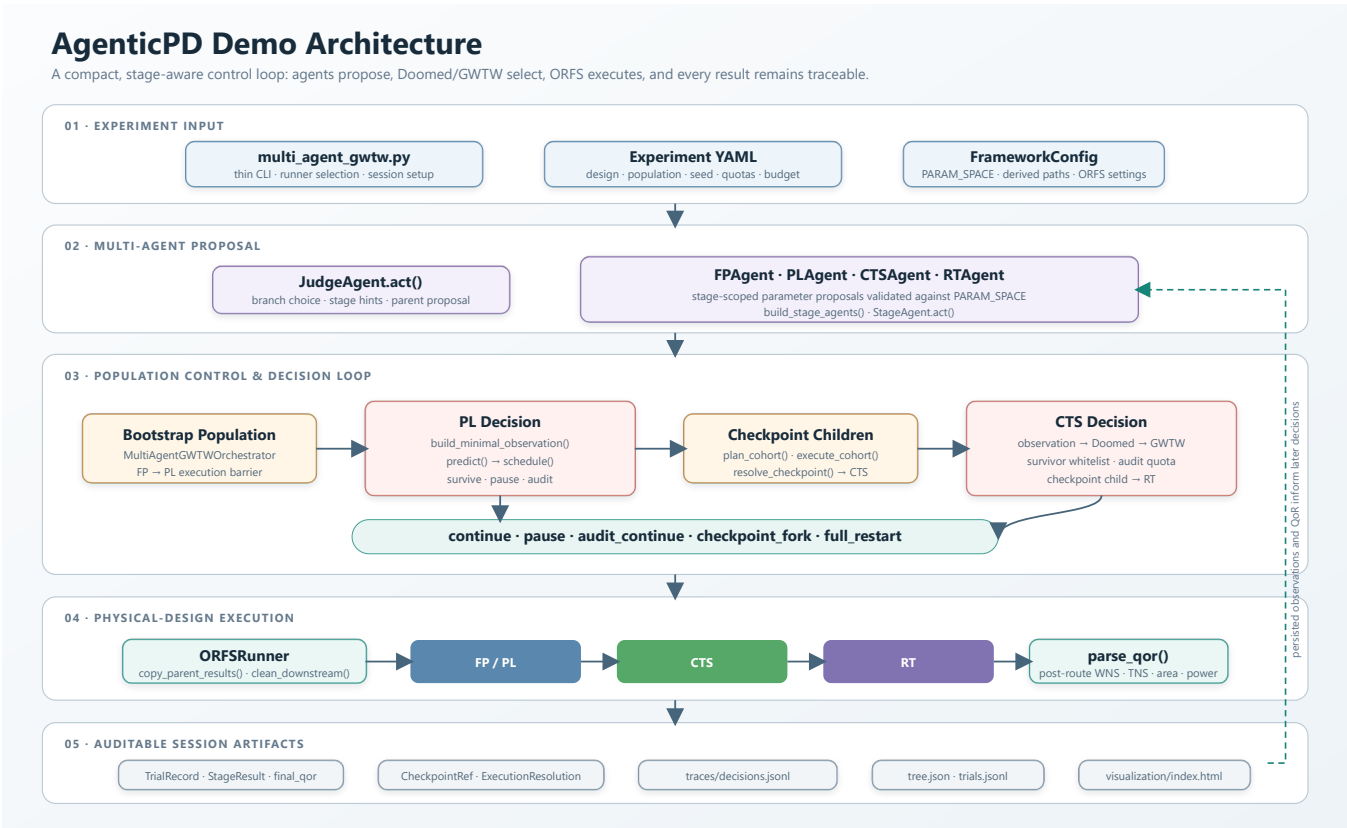
3.3.1 两条执行线



图中上半部分是原生多 Agent iteration；下半部分是 population-based Doomed/GWTW Demo。两条线共享 Agent、ORFS、Trial、checkpoint 和优化树基础，但使用独立入口和不同 orchestration，不通过隐藏开关改变原生优化循环。

3.4 Architecture

3.4.1 Demo 分层结构



架构按单向主链阅读：

1. `multi_agent_gwtw.py` 和 `YAML` 提供实验输入；
2. Judge + 四个 Stage Agent 生成候选；
3. orchestrator 在 PL/CTS 编排 Doomed、GWTW 和 checkpoint child；
4. `ORFSRunner` 执行物理设计阶段，`parse_qor()` 读取最终指标；
5. Trial、trace、tree、checkpoint 和 HTML 构成可审计证据。

4. Data Model 与对外接口

4.1 实验前参数在哪里配置

4.1.1 原生入口配置

原生入口的代码真相源是 `config.py`：

- `PARAM_SPACE`：九个可调参数的 stage、类型、范围、默认值、delivery kind 和 `affects`；
- `BASELINE_PARAMS`：baseline 使用的完整阶段参数；
- `FrameworkConfig`：platform、design、timeout、iteration、LLM、QoR tolerance、variant 和 路径。

CLI 中显式传入的 `--platform`、`--design`、`--iterations`、`--timeout`、`--wns-tol` 和 `--tns-tol` 会覆盖 `FrameworkConfig` 默认值。

4.1.2 Doomed/GWTW Demo 配置

Demo 的唯一运行配置入口是 `configs/experiments/multi-agent-gwtw-demo.yml`，重要字段包括：

- `experiment_id`；
- `design.platform`、`design.design`、`design.role`；
- `evaluator.type` 和 QoR tolerance 声明；
- `population.size`；
- `decision_stages`；
- `decisions.PL/CTS.survivor_count`；
- `audit_quota`、`max_children_per_parent`；
- `seed`；
- `budget.max_trials`、`budget.wall_clock_s`；
- Doomed、scheduler 和 planner 的版本号。

每次正式实验应复制出新的 YAML 并冻结，不能在得到结果后回改原声明。

4.1.3 配置与环境快照

入口创建 session 后写入 `config_snapshot.json`。正式跨环境比较还应刷新 `environment_manifest.json`，记录 ORFS、OpenROAD、PDK、Python 和系统版本。`config_hash` 与 `env_hash` 用于把 Trial 关联到当时的配置和环境证据。

4.2 主要 Data Model

4.2.1 TrialRecord

`schemas/trial.py::TrialRecord` 是一次候选执行的核心记录，重要字段如下：

字段	含义
<code>trial_id</code> 、 <code>experiment_id</code>	8 位随机 Trial 标识和实验标识
<code>parent_trial_id</code> 、 <code>branch_stage</code>	lineage 和分支位置
<code>status</code>	<code>running</code> 、 <code>ok</code> 、 <code>failed</code> 或 <code>paused</code>
<code>params</code> 、 <code>param_diff</code>	完整继承参数及相对 parent 的变化
<code>stage_results</code>	每个阶段的执行记录
<code>final_qor</code>	finish 后 WNS/TNS/Area/Power
<code>failure</code> 、 <code>error_message</code>	失败分类与摘要
<code>checkpoint</code>	本 Trial 产生的 checkpoint
<code>execution_resolution</code>	本 Trial 实际消费 checkpoint 的裁决
<code>doomed_decisions</code> 、 <code>gtw_decisions</code>	PL/CTS 的内联决策
<code>decision_trace_refs</code>	指向 append-only trace 的稳定引用
<code>config_hash</code> 、 <code>env_hash</code>	配置和环境关联
<code>artifact_dir</code>	相对 session 的 Trial 目录

只有 `status == "ok"`、无 failure 且 `final_qor` 完整时，`TrialRecord.is_complete` 才为真。

4.2.2 StageResult

`StageResult` 描述一次 stage 执行，保存：

- `stage`、`status`、`elapsed_s`、`exit_code`；
- 脱敏后的 `command`、`log_path`、`report_path`；
- `start_time`、`end_time`；
- `stage_qor`；
- `failure` 和 `error_message`。

失败 stage 也必须记录耗时和退出码，后续 stage 应标为 `skipped`，而不能把失败 候选从 cohort 中静默丢弃。

4.2.3 CheckpointRef 与 ExecutionResolution

`CheckpointRef` 保存 `checkpoint_id`、source Trial、stage、parameter hash、ORFS revision、artifact directory 和逐文件 SHA-256 manifest。

`ExecutionResolution` 同时保存请求和实际执行结果：

- `requested_parent_node_id`、`requested_start_stage`；

- `effective_start_stage`、`execution_mode`；
- `consumed_checkpoint`、`consumed_node_id`、`consumed_variant`；
- manifest 和参数兼容性结果；
- `fallback_reason`；
- 所有候选 checkpoint 的 `checkpoint_audit_trail`。

这是解释“为何发生 full restart”或“实际复用了哪个祖先 checkpoint”的关键对象。

4.2.4 Observation 与决策对象

`MinimalObservation` 是 `Doomed/GWTW` 的最小输入；`DoomedDecision` 保存 `risk_class`、`risk_score`、`reason_codes`、rule version 和输入证据；`GWTWDecision` 保存 action、decision stage、rank、audit 标记和 scheduler version。PL 和 CTS 决策使用列表保存，避免后一个阶段覆盖前一个阶段。

4.2.5 OptimizationTree 与 DecisionTrace

`OptimizationTree` 的 `OptimNode` 记录 parent/child、stage、iteration、params、QoR、variant 和 `source_trial_id`。它说明搜索关系，但不能替代 checkpoint manifest。

`DecisionTraceWriter` 向 `traces/decisions.jsonl` 追加 `agent_proposal`、`observation`、`doomed_decision`、`gtw_decision`、`parent_selection`、`fork`、`execution_resolution` 和 `cohort_complete` 等事件。`decision_id` 与 `cohort_id` 用于幂等恢复和跨文件关联。

4.3 对外接口

4.3.1 CLI 接口

- `main.py`：原生优化、baseline、parse-only、mock 和 resume。
- `multi_agent_gwtw.py`：YAML 驱动的 `Doomed/GWTW` Demo。
- `tools/session_visualize.py`：从静态 session 生成 HTML。
- `tools/trial_inspect.py`：查询 Trial 和阶段结果。
- `tools/trial_reproduce.py`：按已记录参数重新执行候选。
- `tools/clean.py`：限定 platform/design 的产物清理。

4.3.2 Agent 与 LLM 接口

`BaseAgent.act(context)` 是 Agent 调用入口；`JudgeAgent` 和 `StageAgent` 分别实现 prompt、JSON schema 和 validate。底层只依赖具备 `chat_json(system, user, schema_desc, tag)` 的 client，因此真实 `LLMClient` 和 `MockLLMClient` 可以替换。

4.3.3 ORFS Runner 接口

orchestrator 依赖的窄接口为：

- `run_stage(stage, stage_params, variant, iteration)`；
- `run_finish(stage_params, variant, iteration)`；

- `copy_parent_results(parent_variant, new_variant);`
- `clean_downstream(variant, effective_start)。`

真实 `ORFSRunner` 和 Mock runner 应遵守同一调用语义。正确顺序是 `copy` → `clean downstream` → `run stage` → `finish`。

4.3.4 Manager、Resolver 与纯函数接口

- `TrialManager.create/update/get/list_all()` 管理 Trial 生命周期;
- `CheckpointManager.create/verify/is_compatible/load()` 管理 checkpoint;
- `resolve_checkpoint()` 是实际 parent、checkpoint 和起点的唯一裁决者;
- `build_minimal_observation()`、`predict()`、`schedule()` 和 `plan_cohort()` 是尽量无 I/O 的数
据决策边界;
- `execute_cohort()` 负责把计划转化为可恢复的持久化行为。

4.4 实验完成后结果在哪里

4.4.1 AgenticPD Session

```
runs/<platform>_<design>/<session>/
├─ agenticpd.log
├─ config_snapshot.json
├─ trials.jsonl
├─ tree.json
├─ traces/decisions.jsonl
├─ iter-<N>-<trial_id>/
│   ├─ trial.json
│   └─ checkpoints/<stage>.json
└─ visualization/
    ├─ index.html
    └─ session_data.json
```

`trials.jsonl` 是 append-only index，读取时按 `trial_id` 采用 last-wins；单 Trial 的当前完整状态以其 `trial.json` 为准。

4.4.2 ORFS 原始产物

真实 EDA 产物位于 ORFS `flow/` 下：

```
results/<platform>/<design>/<variant>/
logs/<platform>/<design>/<variant>/
reports/<platform>/<design>/<variant>/
objects/<platform>/<design>/<variant>/
```

最终 QoR 的权威来源是 `reports/.../6_report.json`，文本 report/log 只作为 兼容 fallback。`runs/` 中的摘要和可视化是派生视图，不能替代原始 report。

4.4.3 结果审查顺序

一次实验建议按以下顺序审查：

1. `config_snapshot.json` : 确认 platform、design、seed、预算、mock 标记;
2. `trials.jsonl` : 确认 Trial 总数和最终状态;
3. `trial.json` : 检查参数、stage result、decision 和 resolution;
4. `traces/decisions.jsonl` : 检查 Doomed/GWTW、parent、fork 和 cohort;
5. `tree.json` : 检查 lineage 与 variant;
6. ORFS `6_report.json` : 确认最终真实 QoR;
7. `visualization/index.html` : 辅助理解整体过程。

5. 项目不足与后续改进方向

5.1 当前工程不足

5.1.1 Orchestrator 重复且体积过大

原生 `Optimizer`、组件 `orchestrator` 和 `MultiAgentGWTWOrchestrator` 中存在相似的 Trial 创建、阶段推进、tree 更新、预算和 finish 逻辑。Demo orchestrator 已经超过适合单文件维护的规模。后续应抽取统一的 `TrialExecutionService` 和 `StagePipeline`，但保持原生搜索策略与 population scheduler 解耦。

5.1.2 Demo 缺少正式 resume CLI

底层 cohort executor 已有基于 Trial 和 trace 的幂等重建，但 `multi_agent_gwtw.py` 没有公开 `--resume <session>`。当前重新执行命令会创建新 session，不能作为完整的跨进程恢复入口。后续应从现有 session 加载 YAML snapshot、TrialManager、tree 和 trace，并证明零重复执行。

5.1.3 预算控制仍不完整

`max_trials` 已用于 child 创建前预留；`wall_clock_s` 主要是运行后的检查，不能主动中断正在执行的 ORFS stage。后续需要 deadline 传播、runner cancel、明确的 timeout failure 和恢复语义。LLM token/cost 也尚未纳入统一预算。

5.1.4 测试不随仓库分发

当前 `tests/` 被配置为本地目录且不进 Git。优点是仓库更轻，代价是新 clone 无法复现 489 项本地回归，只能运行模块自检和 `make check`。若项目要交付给他人或形成论文 artifact，应建立可分发的外部测试包、release artifact 或 CI 证据，否则行为契约会随开发环境丢失。

5.1.5 配置入口尚未统一

原生入口主要依赖 `config.py + CLI`，Demo 依赖 YAML。两条线的实验声明方式不同，不利于公平对照。后续应定义统一 ExperimentConfig schema，同时保留不同 orchestrator 的专用策略字段。

5.2 当前算法不足

5.2.1 Doomed 仍是规则分类器

当前 predictor 依据失败状态和 cohort 内 WNS/TNS 排名输出 hard-dead、soft-bad、survivor。它没有使用完整轨迹特征，也没有训练集、概率校准、OOD 检测或跨设计泛化证据。risk_score 不能解释为 doomed probability。

后续可先收集 FP/PL/CTS 特征与最终 finish 标签，再比较规则模型、Logistic Regression、GBDT 和时序模型；只有在 held-out design 上验证后才应替换规则。

5.2.2 GWTW 是简化的配额调度

当前 GWTW 以 survivor count、audit quota 和 population invariant 为核心，尚未实现论文式权重复制、概率重采样或有效样本量分析。后续可将 scheduler 扩展成可插拔 policy，并用同一 trace schema 比较不同 allocation strategy。

5.2.3 缺少性能结论

当前 Demo 证明的是“机制能跑通并产生证据”，不是“Doomed/GWTW 一定提升 QoR 或节省时间”。后续至少需要：

- 多 seed；
- 多 design，且 development 与 held-out 分离；
- 与原生多 Agent、Random Search、无 Doomed/GWTW 的相同预算对照；
- QoR、成功率、真实 wall-clock、LLM token 和被提前停止计算量；
- audit_continue 对误杀率的估计。

5.3 系统与实验不足

5.3.1 设计可迁移性有限

当前九参数空间主要针对 sky130hd/gcd。不同 PDK、宏单元比例和设计规模下，相同参数范围可能没有意义。迁移前必须重新检查 PARAM_SPACE、FastRoute Tcl、QoR 单位和 checkpoint artifact 清单。

5.3.2 并发与集群执行未闭环

当前主要是单机同步执行。SlurmBackend 仍是接口预留，尚未形成作业提交、资源配额、取消、失败重试和日志回收闭环。population 并行化还需要锁、幂等写入和 cohort barrier 的并发安全。

5.3.3 环境溯源与路径脱敏仍需加强

environment_manifest.json 中 unresolved revision 会削弱跨机器比较。此外，部分 snapshot 字段可能记录绝对 YAML 路径。后续应在正式实验前冻结 ORFS/OpenROAD/PDK commit，并统一将可持久化路径表示为相对项目或 session 路径。

5.3.4 可视化是派生视图

HTML 能展示 timeline、cohort、Trial、tree 和 QoR，但它依赖静态 trace 的完整性。可视化不验证 checkpoint manifest，也不能证明 report 未被修改。后续可加入 schema version、缺失证据告警、manifest verification 摘要和原始证据跳转。

5.4 建议的后续优先级

5.4.1 短期：提高 Demo 可靠性

优先完成正式 `--resume`、主动 wall-clock deadline、配置 schema 统一、环境 manifest 冻结和证据路径脱敏。此阶段不急于训练复杂 Doomed 模型。

5.4.2 中期：建立可比较实验

冻结 baseline、预算和 evaluator，加入多个 seed 与至少一个 held-out design，报告 QoR、运行时间、提前停止比例和 audit 误杀率。只有先建立可信对照，算法 升级才有判断依据。

5.4.3 长期：替换策略而非重写执行层

把 DoomedPredictor、GWTWScheduler 和 parent selection 设计成可替换 policy；保持 Trial、checkpoint、ORFSRunner 和 trace 接口稳定。这样可以比较规则、统计模型和学习模型，而不用为每种算法复制一套执行基础设施。

6. 我的理解

AgenticPD本质是依托ORFS平台，对后端物理设计(综合后网表到GDS)进行LLM调参，目前有两路线 一条是复现论文AgenticPD，采用JudgeAgent+4个Stage Agent 另一条是原来基础上引入Doomed Prediction和GWTW算法,在设计早期淘汰掉过差的设计,将迭代预算分配给其它候选;本实现选择了PL和CTS两个检测关卡

1.目前仍然是demo版本,实验已跑通,规模小,且未能说明QoR有真实提升 2.后续应该增大参数空间(版本化,当前9参数为v1.0),且增多实验次数以观察实际效果 3.目前项目在flow/下运行,后续还要接口化,封装成独立的part 4.LLM层未更新,还没有优化提示词